The first line sets up our interrupt pin, although as will be explained in a moment, this number is not the same as the Arduino pin number that our button is connected to. We then set up our analog input, followed by some variables to set the values for hue, saturation, and brightness. Our next variable, which follows, is a little different:

```
volatile int i = 0;
```

As we discussed earlier, a variable's value is not kept inside of a function that uses it, so we would loose track of which button state we were on unless we told the compiler to save this information for us. We've done that here with the `volatile` variable qualifier attached to our index variable, so that we can keep track of the value—hue, saturation, or brightness—we are currently adjusting.

Our setup function is the same as in our last project, except for the addition of the following line:

```
attachInterrupt(buttonInterrupt, selectHSB, RISING);
```

Here is the function that establishes which interrupt we will be using, the function to call when the interrupt is triggered, and what condition will trigger the interrupt. This will be explained shortly.

Our loop function predominately features a `switch… case` statement that will adjust the hue, saturation, or brightness depending on the number of button presses. So, beginning with the `switch` statement and the first case, as follows:

```
switch (i) {
  case 0:
    hue = map(analogRead(analogIn), 0, 1024, 0, 255);
    fadeToHSB(blinkM, hue, 255, 255);
    break;
```

The `switch` statement has been tied to the value of the variable `i`, which will be incremented later in the code. If `i` is equal to 0, then case 0 will execute. This case will read the value of the analog input pin and map that value from a range of 0–1024 to 0–255. This value is then sent to the BlinkM using the `fadeToHSB()` function that will also set the saturation and brightness to their highest setting while we adjust the hue. Finally, we've used the break statement to exit the switch statement without running the other cases.

Cases 1 and 2 will respectively adjust the saturation and brightness, but only when the buttons index had been incremented. Otherwise, for as long as `i` is equal to a value between 0 and 2, then that corresponding case will run. To increment `i`, we use an interrupt service routine, which is a special kind of function, as follows:

```
void selectHSB() {
  ++i %= 4;
}
```

This is the extent of our interrupt service routine. It's one and only job is to increment the index variable `i` by one each time the button is pressed and to keep the numbers to within four possible outcomes, 0, 1, 2, and 3. This line is a little tricky… By placing the `++` in front of the variable `i`, we first increment `i` by one before taking the modulo of `i` and then reassigning the new value back to `i`. This is a little different from the normal `i++` but it's a fairly convenient way to condense a couple lines of code into one.

The rest of the functions are the same BlinkM functions that we used before, so let's skip over these to have a little closer look at how advanced functions work with hardware interrupts.